

13. Релационен модел.

Релационни езици.

Нормализация и

Денормализация на данните.

1. Релационен модел на данните

Релационният модел се базира на математическото понятие за релация (отношение) и множество. Основната идея е данните да се представят под формата на двумерни таблици.

1.1 Основни понятия

- **Домейн** - Произволно множество от неделими (атомарни) стойности. Елементите на домейна са най-малката семантично значима единица от данни. Домейните отговарят на понятието "тип на данните".
 - Примери за домейни:
 - Имена: {Иван, Ана, Александър}
 - Оценка: {2, 3, 4, 5, 6}
- **Атрибут** - Именувана колона на релацията, свързана с определен домейн.
- **Релационна схема** $R(A_1, A_2, A_3, \dots, A_n)$ - Крайно множество от имена на атрибути.
 - Например: `СТУДЕНТ(Фак. Номер, Име, Адрес, Специалност)`
- **Степен на отношението** - Брой атрибути n
 - Нека $D_1, D_2, D_3, \dots, D_n$ са n домейна и $R(A_1, A_2, A_3, \dots, A_n)$ е релационната схема, $D_i = \text{dom}(A_i)$ и $i = 1, 2, 3, \dots, n$.
- **Отношение** r - Произволно подмножество на декартовото произведение на домейните $D_1, D_2, D_3, \dots, D_n$.

- **Кортеж** t - Наредена n -торка от стойности $(v_1, v_2, v_3, \dots, v_n)$.
- **Мощност** m - Броят на кортежите.

Основни свойства на релацията:

- Кортежите в отношението не са наредени
- Няма повтарящи се кортежи (релацията е множество)
- Стойностите в рамките на даден кортеж не са наредени
- Всяка стойност на кортеж е атомарна
- Всеки атрибут има уникално име в рамките на релацията
- Стойностите на даден атрибут принадлежат на един и същ домейн

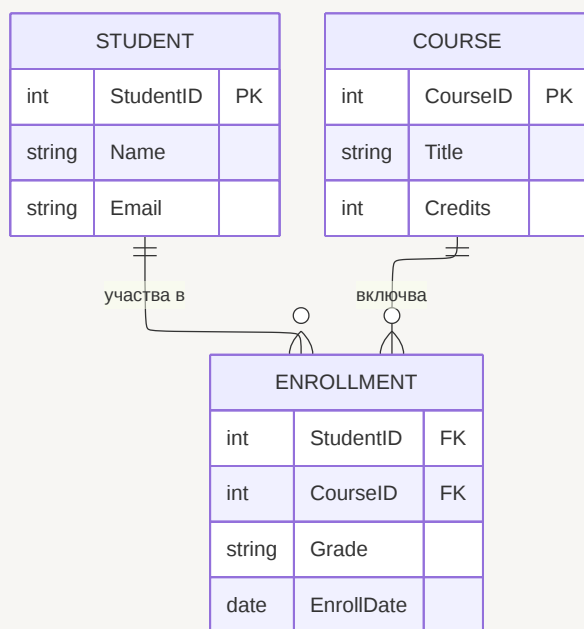
1.2 Ключове

- **Суперключ** - Подмножество от атрибути, което еднозначно идентифицира кортеж
 - Пример: {Име, Фамилия, ЕГН}
- **Кандидат-ключ** - Минимален суперключ (суперключ, от който не може да се премахне нито един атрибут, без да се загуби свойството за уникалност)
- **Първичен ключ (Primary Key, PK)** - Кандидат-ключ, който е избран като основен идентификатор на кортежите.
- **Външен ключ (Foreign key, FK)** - Атрибут или множество от атрибути в една релация, чиито стойности съвпадат към първичния ключ на друга (или същата) релация и реализира връзка между тях.

1.3 Релационни ограничения

1. **Ограничения на домейна** - Всички стойности в дадена колона трябва да принадлежат към дефинирания за нея домейн.
2. **Ограничения върху ключа и нулевите стойности**

- a. Всички записи (кортежи) са уникални в релацията. В една релация не може да има два кортежа (реда), които да имат едни и същи стойности за множеството от атрибути, дефинирани като ключ (суперключ или кандидат-ключ).
 - b. Ограниченията върху нулевите стойности определят дали е допустимо да няма въведена стойност (NULL) за даден атрибут.
3. **Ограничения за цялостност на обекта** - Атрибутите от първичния ключ не могат да съдържат NULL.
4. **Ограничения за цялостност на връзките** - Всяка стойност на външен ключ трябва или да съответства на съществуваща стойност на първичен ключ в свързаната релация, или да бъде NULL.



2. Релационни езици

Релационните езици служат за манипулиране на данните в релационния модел. Те се делят на процедурни (релационна алгебра) и декларативни (SQL).

2.1 Релационна алгебра

Теоретичен език с набор от операции, които приемат една или две релации като вход и връщат нова релация като резултат. Тя описва как да се извлекат данните.

Основни операции:

- **Селекция (σ):** Извлича подмножество от кортежи (редове), които отговарят на зададено логическо условие. Формат: $\sigma_{condition}(R)$.
- **Проекция (π):** Извлича подмножество от атрибути (колони), като премахва дублиращите се редове в резултата. Формат: $\pi_{A_1, A_2, \dots, A_n}(R)$.
- **Съединение ($\bowtie$):** Комбинация от Декартово произведение и последваща Селекция, базирана на общи атрибути.
- **Обединение ($\cup$):** Събира кортежите на две релации, които трябва да са съвместими по обединение (еднакъв брой и тип атрибути).
- **Сечение ($\cap$):** Връща кортежите, присъстващи едновременно и в двете релации.
- **Разлика ($-$):** Връща кортежите, които са в първата релация, но не са във втората.

2.2 Релационно смятане

Декларативен език - описва какво се търси, а не как да се извърши.

- **Релационно смятане, базирано на кортежи**
 - Променливите приемат като стойности цели кортежи.
- **Релационно смятане, базирано на домейни**
 - Променливите приемат стойности от домейните на атрибутите.

3. Нормализация на данните

Нормализацията е процес на проектиране и реструктуриране на релационната база от данни с цел минимизиране на излишъка от данни и предотвратяване на аномалии при модификация.

3.1 Аномалии

Пример - ненормализирана таблица:

StudentID	CourseID	CourseName	Instructor
101	C1	Бази от данни	Иванов
101	C2	Алгоритми	Петров
102	C1	Бази от данни	Иванов

Неправилно проектираната схема води до следните проблеми:

- **Аномалия при вмъкване** - Невъзможно е да се добави факт за дадена същност, докато не се създаде факт за друга такава.
 - Не може да се създаде курс без да имаме записан за него студент.
- **Аномалия при обновяване** - Ако един факт е записан на много места, промяната му изисква обновяване на всички тези места. Пропуск води до неконсистентност.
 - Смяната на преподавател на курс C1 води до смяната му в няколко кортежа, което може да доведе до неконсистентност.
- **Аномалия при изтриване** - Изтриването на факт за една същност води до нежелана загуба на информация за друга същност.
 - Изтриването на студент 102, премахва и информацията за курса по "База от данни", ако той не е записан никъде.

3.2 Функционална зависимост

В основата на нормализацията стои функционалната зависимост.

Тя се бележи като $X \rightarrow Y$, което означава, че атрибутът Y е функционално зависим от множеството атрибути X . Ако знаем стойността на X , можем еднозначно да определим стойността на Y .

- **Пълна зависимост** - Y зависи от целия X , а не от негово подмножество.

- **Частична зависимост** - Y зависи само от част от съставен ключ.
- **Транзитивна зависимост** - ако $X \rightarrow Y$ и $Y \rightarrow Z$, то $X \rightarrow Z$ (през незов атрибут).

3.3 Нормални форми

- **Първа нормална форма** - Една релация е в 1NF, ако всички нейни атрибути съдържат само атомарни (неделими) стойности. Забранени са повтарящи се групи или масиви в колоните.
 - Пример нарушение: таблица `Student(ID, Name, Phones)`, където `Phones = "0888123456, 0899654321"` в едно поле.
Решение – извеждане на телефоните в отделна таблица `StudentPhone(StudentID, Phone)`.
- **Втора нормална форма** - Релацията е в 1NF и всеки **неключов атрибут** е напълно функционално зависим от целия първичен ключ (няма частични зависимости от съставен първичен ключ).
 - Пример: `Order(OrderID, ProductID, ProductName, Quantity)` с `PK {OrderID, ProductID}`. `ProductName` зависи само от `ProductID` (частична зависимост) → нарушение на 2НФ.
Декомпозиция:
 - `Order(OrderID, ProductID, Quantity)`
 - `Product(ProductID, ProductName)`
- **Трета нормална форма** - Релацията е в 2NF и нито един **неключов атрибут** не е транзитивно зависим от първичния ключ. Всички неключови атрибути зависят само и единствено от първичния ключ.
 - Пример: `Employee(EmpID, DeptID, DeptName)`, където $EmpID \rightarrow DeptID \rightarrow DeptName$ → транзитивна зависимост, нарушение на 3НФ.
Декомпозиция:
 - `Employee(EmpID, DeptID)`
 - `Department(DeptID, DeptName)`

- **Нормална форма на Бойс-Код (BCNF)** - По-строга версия на 3NF. Една релация е в BCNF тогава и само тогава, когато за всяка нетривиална функционална зависимост $X \rightarrow Y$, X е суперключ.

- *Класически пример:* `Enroll(Student, Course, Instructor)` с FD-та: $\{Student, Course\} \rightarrow Instructor$ и $Instructor \rightarrow Course$ (всеки преподавател води само един курс). Втората зависимост нарушава BCNF, защото `Instructor` не е суперключ.

Декомпозиция:

- `TeachesCourse(Instructor, Course)`
- `StudentInstructor(Student, Instructor)`

Съществуват и по-високи нормални форми (4NF, 5NF, 6NF), свързани с многозначни зависимости и зависимости на съединението, но в практиката дизайнът най-често се довежда до 3NF или BCNF.

4. Денормализация на данните

Въпреки че нормализацията решава проблемите с аномалиите, разбиването на данните в много на брой логически таблици води до нуждата от множество скъпи операции за съединение (JOIN) при извличане. Денормализацията е обратният процес.

4.1 Същност на процеса

Денормализацията е съзнателно, планирано въвеждане на контролиран излишък от данни в схемата на вече нормализирана база данни. Това е стъпка от физическия дизайн на базата данни (а не от логическия).

4.2 Цели

- Оптимизация на четенето като се намали времето при операции като `SELECT`, като се елиминират сложни `JOIN` операции.
- Предварително изчисляване - Често изисквани агрегации се пресмятат и съхраняват като директни атрибути, вместо да се изчисляват при всяка заявка.

4.3 Кога се прилага

- Хранилища за данни и OLAP системи, отчети (reporting)
- Системи с преобладаващо четене (read-heavy) и скъпи многотаблични съединения
- Кеширане на често изчислявани агрегати

4.4 Компромиси

Плюсове	Минуси
По-бързи заявки за четене	По-голямо заемано пространство
По-малко и по-прости JOIN-и	Риск от несъгласуваност при обновяване
По-добра производителност при отчети	Нужда от допълнителна логика (тригери) за поддържане на консистентност